

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №2

По дисциплине «Модели решения задач в интеллектуальных системах»

Тема: «Многослойный персептрон для задачи «исключающее ИЛИ» с числовыми классами: сравнение функций потерь MSE и Binary Cross-Entropy»

Выполнил:

Студент 4 курса

Группы ИИ-26

Яшков П. А.

Проверил:

Никитин С. К.

Цель работы: На основе многослойного персептрона архитектуры 2-2-1, реализованного в Лабораторной работе №1 для задачи XOR с числовыми классами c_0 и c_1 из диапазона $[-10; 10]$, реализовать альтернативную функцию потерь – бинарную кросс-энтропию (Binary Cross-Entropy, BCE) – и провести сравнительный анализ обучения сети с MSE (Лабораторная работа №1) и с BCE по скорости сходимости, качеству обучения и устойчивости к выбору начальных весов.

Ход работы:

```
import numpy as np
import matplotlib.pyplot as plt

# =====
# Лабораторная работа №2
# Вариант 10:  $c_0 = -3$ ,  $c_1 = -8$ 
# Многослойный персептрон 2-2-1
# Сравнение MSE и Binary Cross-Entropy
#
# Конфигурация A:
# MSE считается в исходной шкале  $[-3; -8]$ .
#
# Конфигурация B:
# BCE обучается по бинарным меткам 0/1.
#
# Нормализация:
#  $c_0 = -3 \rightarrow 0$ 
#  $c_1 = -8 \rightarrow 1$ 
#
# Обратное преобразование:
#  $y_{\text{real}} = c_0 + y_{\text{norm}} * (c_1 - c_0)$ 
#  $y_{\text{real}} = -3 - 5 * y_{\text{norm}}$ 
# =====

# -----
# 1. Исходные данные варианта 10
# -----

C0 = -3.0
C1 = -8.0

X_REAL = np.array([
    [-3.0, -3.0],
    [-3.0, -8.0],
    [-8.0, -3.0],
    [-8.0, -8.0]
], dtype=float)

Y_REAL = np.array([
    [-3.0],
    [-8.0],
    [-8.0],
    [-3.0]
], dtype=float)

# Для варианта 10 MSE в исходной шкале имеет более крупный
# градиент, поскольку  $|c_1 - c_0| = 5$ , поэтому используем
# умеренный шаг обучения.
LEARNING_RATE = 0.1

EE_MSE = 0.01
EE_BCE = 0.03
```

```
MAX_EPOCHS = 20000
```

```
# 5 различных начальных инициализаций.
```

```
# Этот набор даёт наглядную серию запусков для сравнения.
```

```
SEEDS = [1, 4, 5, 6, 7]
```

```
SHUFFLE_SEED = 42
```

```
# -----
```

```
# 2. Масштабирование
```

```
# -----
```

```
def to_binary_scale(value, c0=C0, c1=C1):
```

```
    """
```

```
    Перевод исходной шкалы в [0; 1].
```

```
    Для варианта 10:
```

```
    -3 -> 0
```

```
    -8 -> 1
```

```
    """
```

```
    return (value - c0) / (c1 - c0)
```

```
def to_real_scale(y_norm, c0=C0, c1=C1):
```

```
    """
```

```
    Обратный перевод из [0;1] в исходную шкалу.
```

```
    y_real = c0 + y_norm * (c1 - c0)
```

```
    Для варианта 10:
```

```
    y_real = -3 - 5*y_norm
```

```
    """
```

```
    return c0 + y_norm * (c1 - c0)
```

```
# Масштабируем входы одинаково для обеих конфигураций:
```

```
# (-3 -> 0), (-8 -> 1)
```

```
X = to_binary_scale(X_REAL)
```

```
# Бинарные целевые значения:
```

```
# -3 -> 0
```

```
# -8 -> 1
```

```
Y_BINARY = to_binary_scale(Y_REAL)
```

```
# -----
```

```
# 3. Вспомогательные функции
```

```
# -----
```

```
def sigmoid(z):
```

```
    z = np.clip(z, -60.0, 60.0)
```

```
    return 1.0 / (1.0 + np.exp(-z))
```

```
def sigmoid_derivative_from_output(y):
```

```
    return y * (1.0 - y)
```

```
def mse_real_loss(y_true_real, y_pred_norm):
```

```
    """
```

```
    MSE в исходной шкале [-3; -8].
```

```

Es = 1/2 * sum((y_pred_real - y_true_real)^2)
"""

y_pred_real = to_real_scale(y_pred_norm)
return 0.5 * np.sum(
    (y_pred_real - y_true_real) ** 2
)

def bce_loss(y_true_binary, y_pred_norm):
    """
    Binary Cross-Entropy для бинарных меток 0/1.
    """
    eps = 1e-12

    y_pred_norm = np.clip(
        y_pred_norm,
        eps,
        1.0 - eps
    )

    return -np.sum(
        y_true_binary * np.log(y_pred_norm)
        + (1.0 - y_true_binary) * np.log(1.0 - y_pred_norm)
    )

def nearest_class_from_real(y_real):
    """
    Определение ближайшего исходного класса.
    """
    d0 = np.abs(y_real - C0)
    d1 = np.abs(y_real - C1)

    return np.where(
        d0 <= d1,
        C0,
        C1
    )

def classification_accuracy(y_true_real, y_pred_norm):
    y_pred_real = to_real_scale(y_pred_norm)

    true_classes = nearest_class_from_real(
        y_true_real
    )

    pred_classes = nearest_class_from_real(
        y_pred_real
    )

    return np.mean(
        true_classes == pred_classes
    )

def mean_absolute_error_real(y_true_real, y_pred_norm):
    y_pred_real = to_real_scale(y_pred_norm)

    return np.mean(
        np.abs(
            y_true_real - y_pred_real
        )
    )

```

```
# -----  
# 4. MLP 2-2-1  
# -----
```

```
class MLP221:
```

```
    def __init__(self, seed=1):  
  
        rng = np.random.RandomState(seed)  
  
        # Малые случайные начальные веса  
        self.W1 = rng.uniform(  
            -0.1, 0.1,  
            size=(2, 2)  
        )  
  
        self.b1 = rng.uniform(  
            -0.1, 0.1,  
            size=(1, 2)  
        )  
  
        self.W2 = rng.uniform(  
            -0.1, 0.1,  
            size=(2, 1)  
        )  
  
        self.b2 = rng.uniform(  
            -0.1, 0.1,  
            size=(1, 1)  
        )
```

```
    def forward(self, x):  
  
        net1 = x @ self.W1 + self.b1  
        hidden = sigmoid(net1)  
  
        net2 = hidden @ self.W2 + self.b2  
        output_norm = sigmoid(net2)  
  
        return hidden, output_norm
```

```
    def predict_norm(self, x):  
        return self.forward(x)[1]
```

```
    def train(  
        self,  
        x_train,  
        y_real_train,  
        y_binary_train,  
        loss_name,  
        learning_rate,  
        ee,  
        max_epochs,  
        shuffle_seed  
    ):  
  
        rng_shuffle = np.random.RandomState(  
            shuffle_seed  
        )  
  
        history = []  
        converged = False
```

```

for epoch in range(
    1,
    max_epochs + 1
):

    indices = np.arange(
        len(x_train)
    )

    rng_shuffle.shuffle(
        indices
    )

    # Онлайн-режим:
    # веса обновляются после каждого примера.
    for i in indices:

        x = x_train[
            i:i + 1
        ]

        target_real = y_real_train[
            i:i + 1
        ]

        target_binary = y_binary_train[
            i:i + 1
        ]

        # -----
        # Forward
        # -----

        hidden, output_norm = (
            self.forward(x)
        )

        # -----
        # Градиент выходного слоя
        # -----

        if loss_name == "mse":

            output_real = (
                to_real_scale(
                    output_norm
                )
            )

            #  $E = 1/2 * (y_{real} - target)^2$ 
            #
            #  $y_{real} =$ 
            #  $c_0 + y_{norm} * (c_1 - c_0)$ 
            #
            #  $dE/dnet =$ 
            #  $(y_{real} - target)$ 
            #  $* (c_1 - c_0)$ 
            #  $* sigmoid'(net)$ 

            delta2 = (
                (output_real - target_real)
                * (C1 - C0)
                * sigmoid_derivative_from_output(

```

```

        output_norm
    )
)

elif loss_name == "bce":

    # BCE + sigmoid:
    #
    #  $dE/dnet = y\_hat - y$ 
    delta2 = (
        output_norm
        - target_binary
    )

else:
    raise ValueError(
        "loss_name должен быть "
        "'mse' или 'bce'"
    )

# -----
# Градиент скрытого слоя
# -----

delta1 = (
    (delta2 @ self.W2.T)
    * sigmoid_derivative_from_output(
        hidden
    )
)

# -----
# Обновление параметров
# -----

self.W2 -= (
    learning_rate
    * hidden.T
    @ delta2
)

self.b2 -= (
    learning_rate
    * delta2
)

self.W1 -= (
    learning_rate
    * x.T
    @ delta1
)

self.b1 -= (
    learning_rate
    * delta1
)

# -----
# Ошибка после эпохи
# -----

predictions_norm = (
    self.predict_norm(
        x_train

```

```

    )
)

if loss_name == "mse":

    error = mse_real_loss(
        y_real_train,
        predictions_norm
    )

else:

    error = bce_loss(
        y_binary_train,
        predictions_norm
    )

history.append(
    float(error)
)

if error <= ee:

    converged = True
    break

return {
    "epochs": epoch,
    "final_error": float(error),
    "converged": converged,
    "history": history
}

```

```

# -----
# 5. Один запуск
# -----

```

```

def run_one_configuration(
    loss_name,
    seed
):

    model = MLP221(
        seed=seed
    )

    if loss_name == "mse":
        ee = EE_MSE
    else:
        ee = EE_BCE

    training = model.train(
        x_train=X,
        y_real_train=Y_REAL,
        y_binary_train=Y_BINARY,
        loss_name=loss_name,
        learning_rate=LEARNING_RATE,
        ee=ee,
        max_epochs=MAX_EPOCHS,
        shuffle_seed=SHUFFLE_SEED
    )

    pred_norm = model.predict_norm(

```

```

        X
    )

    pred_real = to_real_scale(
        pred_norm
    )

    accuracy = classification_accuracy(
        Y_REAL,
        pred_norm
    )

    mae = mean_absolute_error_real(
        Y_REAL,
        pred_norm
    )

    return {
        "loss_name": loss_name,
        "seed": seed,
        "model": model,

        "epochs": training["epochs"],
        "final_error": training["final_error"],
        "converged": training["converged"],
        "history": training["history"],

        "pred_norm": pred_norm,
        "pred_real": pred_real,

        "accuracy": float(
            accuracy
        ),

        "mae": float(
            mae
        )
    }

```

```

# -----
# 6. Серия из пяти запусков
# -----

```

```

def run_experiments():

```

```

    mse_runs = []
    bce_runs = []

```

```

    print(
        "\n"
        + "=" * 78
    )

```

```

    print(
        "ЛАБОРАТОРНАЯ РАБОТА №2 "
        "— ВАРИАНТ 10"
    )

```

```

    print(
        "c0 = -3, c1 = -8"
    )

```

```

    print(

```

```

"=" * 78
)

print(
    "\nКонфигурация А: "
    "MSE в исходной шкале [-3; -8]"
)

for seed in SEEDS:

    result = (
        run_one_configuration(
            "mse",
            seed
        )
    )

    mse_runs.append(
        result
    )

    status = (
        "ДА"
        if result["converged"]
        else "НЕТ"
    )

    print(
        f"seed={seed:2d} | "
        f"эпох={result['epochs']:5d} | "
        f"Es={result['final_error']:.6f} | "
        f"accuracy={result['accuracy']:.2f} | "
        f"MAE={result['mae']:.6f} | "
        f"сходимость={status}"
    )

print(
    "\nКонфигурация В: "
    "BCE по нормализованным целям 0/1"
)

for seed in SEEDS:

    result = (
        run_one_configuration(
            "bce",
            seed
        )
    )

    bce_runs.append(
        result
    )

    status = (
        "ДА"
        if result["converged"]
        else "НЕТ"
    )

    print(
        f"seed={seed:2d} | "
        f"эпох={result['epochs']:5d} | "
        f"Es={result['final_error']:.6f} | "

```

```

        f"accuracy={result['accuracy']:.2f} | "
        f"MAE={result['mae']:.6f} | "
        f"сходимость={status}"
    )

    return (
        mse_runs,
        bce_runs
    )

# -----
# 7. Представительный запуск
# -----

def choose_representative_run(
    runs
):

    successful = [
        run
        for run in runs
        if run["converged"]
    ]

    if successful:

        epochs = np.array(
            [
                run["epochs"]
                for run in successful
            ]
        )

        median_epochs = (
            np.median(
                epochs
            )
        )

        return min(
            successful,
            key=lambda run:
                abs(
                    run["epochs"]
                    - median_epochs
                )
        )

    return min(
        runs,
        key=lambda run:
            run["final_error"]
    )

# -----
# 8. Таблица предсказаний
# -----

def print_prediction_table(
    title,
    run
):
```

```

print(
    "\n" + title
)

print(
    "_" * 88
)

print(
    f"{'A':>7}"
    f"{'B':>8}"
    f"{'Цель':>10}"
    f"{'ŷ norm':>14}"
    f"{'ŷ real':>14}"
    f"{'Класс':>12}"
)

print(
    "_" * 88
)

for (
    x_real,
    target_real,
    pred_norm,
    pred_real
) in zip(
    X_REAL,
    Y_REAL.ravel(),
    run["pred_norm"].ravel(),
    run["pred_real"].ravel()
):

    predicted_class = (
        C0
        if abs(
            pred_real - C0
        )
        <= abs(
            pred_real - C1
        )
        else C1
    )

    print(
        f"{x_real[0]:7.1f}"
        f"{x_real[1]:8.1f}"
        f"{target_real:10.1f}"
        f"{pred_norm:14.4f}"
        f"{pred_real:14.4f}"
        f"{predicted_class:12.1f}"
    )

# -----
# 9. Сводка по пяти запускам
# -----

def print_summary(
    mse_runs,
    bce_runs
):

```

```

print(
    "\n"
    + "=" * 78
)

print(
    "СВОДКА ПО 5 ЗАПУСКАМ"
)

print(
    "=" * 78
)

for name, runs in [
    ("MSE", mse_runs),
    ("BCE", bce_runs)
]:

    successful = [
        run
        for run in runs
        if run["converged"]
    ]

    print(
        f"\n{name}:"
    )

    print(
        f" Достигли критерия: "
        f"{len(successful)} "
        f"из {len(runs)}"
    )

    if successful:

        epochs = np.array(
            [
                run["epochs"]
                for run in successful
            ]
        )

        print(
            f" Минимум эпох:  "
            f"{epochs.min()}"
        )

        print(
            f" Максимум эпох:  "
            f"{epochs.max()}"
        )

        print(
            f" Среднее эпох:  "
            f"{epochs.mean():.2f}"
        )

        print(
            f" Разброс:  "
            f"{epochs.max() - epochs.min()}"
        )

    mean_accuracy = np.mean(

```

```

    [
        run["accuracy"]
        for run in runs
    ]
)

mean_mae = np.mean(
    [
        run["mae"]
        for run in runs
    ]
)

print(
    f" Средняя accuracy: "
    f"{mean_accuracy:.4f}"
)

print(
    f" Средняя MAE: "
    f"{mean_mae:.6f}"
)

# -----
# 10. График сходимости
# -----

def plot_convergence(
    mse_run,
    bce_run
):

    plt.figure(
        figsize=(10, 6)
    )

    mse_epochs = np.arange(
        1,
        len(
            mse_run["history"]
        ) + 1
    )

    bce_epochs = np.arange(
        1,
        len(
            bce_run["history"]
        ) + 1
    )

    plt.plot(
        mse_epochs,
        mse_run["history"],
        label=(
            f"MSE, seed="
            f"{mse_run['seed']}"
        )
    )

    plt.plot(
        bce_epochs,
        bce_run["history"],
        label=(

```

```

        f"BCE, seed="
        f"{bce_run['seed']}"
    )
)

plt.axhline(
    EE_MSE,
    linestyle="--",
    label=(
        f"Ee MSE = "
        f"{EE_MSE}"
    )
)

plt.axhline(
    EE_BCE,
    linestyle=":",
    label=(
        f"Ee BCE = "
        f"{EE_BCE}"
    )
)

plt.yscale(
    "log"
)

plt.xlabel(
    "Номер эпохи"
)

plt.ylabel(
    "Суммарная ошибка Es"
)

plt.title(
    "Сравнение сходимости "
    "MSE и BCE — вариант 10"
)

plt.grid(
    True,
    alpha=0.3
)

plt.legend()
plt.tight_layout()

# -----
# 11. Диаграмма эпох по seed
# -----

def plot_epochs_by_seed(
    mse_runs,
    bce_runs
):

    x = np.arange(
        len(SEEDS)
    )

    width = 0.36

```

```

mse_epochs = [
    run["epochs"]
    for run in mse_runs
]

bce_epochs = [
    run["epochs"]
    for run in bce_runs
]

plt.figure(
    figsize=(10, 6)
)

plt.bar(
    x - width / 2,
    mse_epochs,
    width,
    label="MSE"
)

plt.bar(
    x + width / 2,
    bce_epochs,
    width,
    label="BCE"
)

# Если какой-либо запуск не достиг порога,
# он отмечается крестиком.
for i, run in enumerate(
    mse_runs
):
    if not run["converged"]:
        plt.scatter(
            x[i] - width / 2,
            run["epochs"],
            marker="x",
            s=100,
            linewidths=2
        )

for i, run in enumerate(
    bce_runs
):
    if not run["converged"]:
        plt.scatter(
            x[i] + width / 2,
            run["epochs"],
            marker="x",
            s=100,
            linewidths=2
        )

plt.xticks(
    x,
    [
        str(seed)
        for seed in SEEDS
    ]
)

```

```

    )

    plt.xlabel(
        "Seed"
    )

    plt.ylabel(
        "Количество эпох"
    )

    plt.title(
        "Устойчивость сходимости "
        "по 5 случайным инициализациям "
        "— вариант 10"
    )

    plt.legend()

    plt.grid(
        axis="y",
        alpha=0.3
    )

    plt.tight_layout()

# -----
# 12. Разделяющая поверхность
# -----

def plot_decision_surface(
    run,
    title
):

    model = run["model"]

    a_values = np.linspace(
        -10.0,
        10.0,
        250
    )

    b_values = np.linspace(
        -10.0,
        10.0,
        250
    )

    aa, bb = np.meshgrid(
        a_values,
        b_values
    )

    grid_real = np.column_stack(
        [
            aa.ravel(),
            bb.ravel()
        ]
    )

    grid_scaled = (
        to_binary_scale(
            grid_real

```

```

    )
)

z_norm = (
    model.predict_norm(
        grid_scaled
    )
    .reshape(
        aa.shape
    )
)

plt.figure(
    figsize=(8, 7)
)

contour = plt.contourf(
    aa,
    bb,
    z_norm,
    levels=np.linspace(
        0.0,
        1.0,
        21
    ),
    cmap="viridis"
)

plt.colorbar(
    contour,
    label=(
        "Нормализованный "
        "выход  $\hat{y}$ "
    )
)

# Граница между классами:
# y_norm = 0.5
plt.contour(
    aa,
    bb,
    z_norm,
    levels=[0.5],
    linewidths=2
)

true_classes = (
    Y_BINARY
    .ravel()
    .astype(int)
)

plt.scatter(
    X_REAL[:, 0],
    X_REAL[:, 1],
    c=true_classes,
    cmap="coolwarm",
    edgecolors="black",
    s=120,
    label="Обучающие примеры"
)

plt.xlim(
    -10,

```

```

    10
)

plt.ylim(
    -10,
    10
)

plt.xlabel(
    "A"
)

plt.ylabel(
    "B"
)

plt.title(
    title
)

plt.legend()

plt.grid(
    True,
    alpha=0.2
)

plt.tight_layout()

# -----
# 13. Сравнение MAE
# -----

def plot_mae_comparison(
    mse_run,
    bce_run
):

    names = [
        "MSE",
        "BCE"
    ]

    values = [
        mse_run["mae"],
        bce_run["mae"]
    ]

    plt.figure(
        figsize=(7, 5)
    )

    bars = plt.bar(
        names,
        values
    )

    plt.ylabel(
        "Средняя абсолютная ошибка"
    )

    plt.title(
        "MAE представительных запусков "

```

```

        "в исходной шкале [-3; -8]"
    )

plt.grid(
    axis="y",
    alpha=0.3
)

for bar, value in zip(
    bars,
    values
):

    plt.text(
        bar.get_x()
        + bar.get_width() / 2,
        bar.get_height(),
        f"{value:.5f}",
        ha="center",
        va="bottom"
    )

plt.tight_layout()

```

```

# -----
# 14. Режим функционирования
# -----

```

```

def infer_one(
    model,
    a,
    b
):

    x_real = np.array(
        [[a, b]],
        dtype=float
    )

    x_scaled = (
        to_binary_scale(
            x_real
        )
    )

    y_norm = float(
        model.predict_norm(
            x_scaled
        )[0, 0]
    )

    y_real = float(
        to_real_scale(
            y_norm
        )
    )

    nearest_class = (
        C0
        if abs(
            y_real - C0
        )
        <= abs(

```

```

        y_real - C1
    )
    else C1
)

return (
    y_norm,
    y_real,
    nearest_class
)

def demonstrate_functioning(
    model
):

    print(
        "\n"
        + "=" * 78
    )

    print(
        "РЕЖИМ ФУНКЦИОНИРОВАНИЯ "
        + "— ВСЕ — ВАРИАНТ 10"
    )

    print(
        "=" * 78
    )

    # 4 обучающих + 3 дополнительных примера
    examples = [
        (-3.0, -3.0),
        (-3.0, -8.0),
        (-8.0, -3.0),
        (-8.0, -8.0),

        (-5.5, -5.5),
        (-2.0, -7.0),
        (-9.0, -4.0)
    ]

    for a, b in examples:

        (
            y_norm,
            y_real,
            nearest
        ) = infer_one(
            model,
            a,
            b
        )

        print(
            f"A={a:6.2f}, "
            f"B={b:6.2f} | "
            f"ŷ={y_norm:.4f} | "
            f"y_real={y_real:.4f} | "
            f"ближайший класс="
            f"{nearest:.1f}"
        )

```

```

def interactive_mode(
    model
):

    print(
        "\n"
        + "=" * 78
    )

    print(
        "ИНТЕРАКТИВНЫЙ РЕЖИМ"
    )

    print(
        "Введите A и B "
        "из диапазона [-10; 10]."
    )

    print(
        "Для выхода введите q."
    )

    print(
        "=" * 78
    )

    while True:

        text = input(
            "\nA = "
        ).strip()

        if text.lower() == "q":
            break

        try:

            a = float(
                text
            )

            b = float(
                input(
                    "B = "
                ).strip()
            )

        except ValueError:

            print(
                "Ошибка: "
                "нужно ввести число."
            )

            continue

        if not (
            -10.0 <= a <= 10.0
            and
            -10.0 <= b <= 10.0
        ):

            print(
                "Ошибка: A и B должны "

```

```

        "быть в диапазоне "
        "[-10; 10]."
    )

    continue

(
    y_norm,
    y_real,
    nearest
) = infer_one(
    model,
    a,
    b
)

print(
    f"Нормализованный "
    f"выход  $\hat{y}$  = "
    f"{y_norm:.4f}"
)

print(
    f"Выход в исходной шкале "
    f"[{C0:.0f}; {C1:.0f}] "
    f"= {y_real:.4f}"
)

print(
    f"Ближайший класс "
    f"= {nearest:.1f}"
)

# -----
# 15. Главная функция
# -----

def main():

    mse_runs, bce_runs = (
        run_experiments()
    )

    mse_rep = (
        choose_representative_run(
            mse_runs
        )
    )

    bce_rep = (
        choose_representative_run(
            bce_runs
        )
    )

    print_summary(
        mse_runs,
        bce_runs
    )

    print_prediction_table(
        "Представительный запуск MSE",
        mse_rep
    )

```

```

)

print_prediction_table(
    "Представительный запуск BCE",
    bce_rep
)

demonstrate_functioning(
    bce_rep["model"]
)

# Все требуемые графики
plot_convergence(
    mse_rep,
    bce_rep
)

plot_epochs_by_seed(
    mse_runs,
    bce_runs
)

plot_decision_surface(
    mse_rep,
    (
        "Разделяющая поверхность "
        f"MSE, seed="
        f"{mse_rep['seed']}"
        "— вариант 10"
    )
)

plot_decision_surface(
    bce_rep,
    (
        "Разделяющая поверхность "
        f"BCE, seed="
        f"{bce_rep['seed']}"
        "— вариант 10"
    )
)

plot_mae_comparison(
    mse_rep,
    bce_rep
)

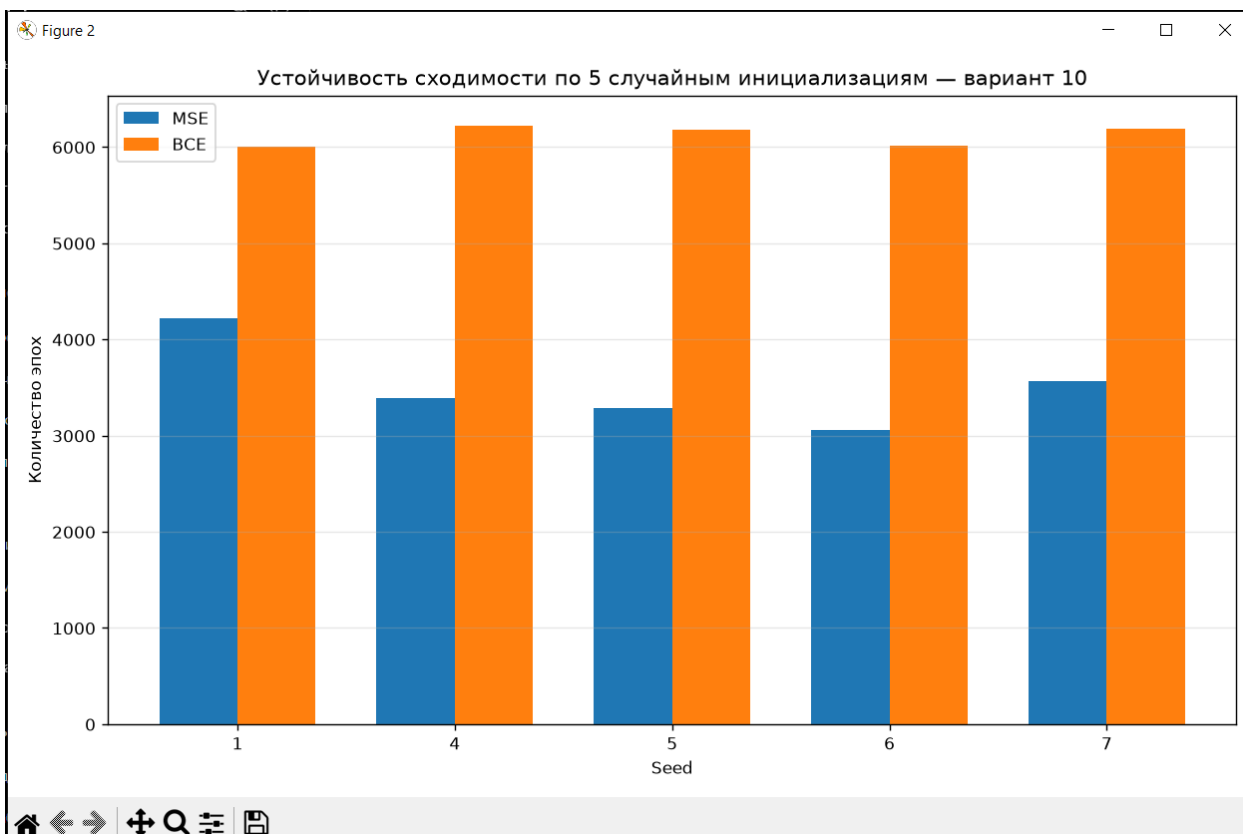
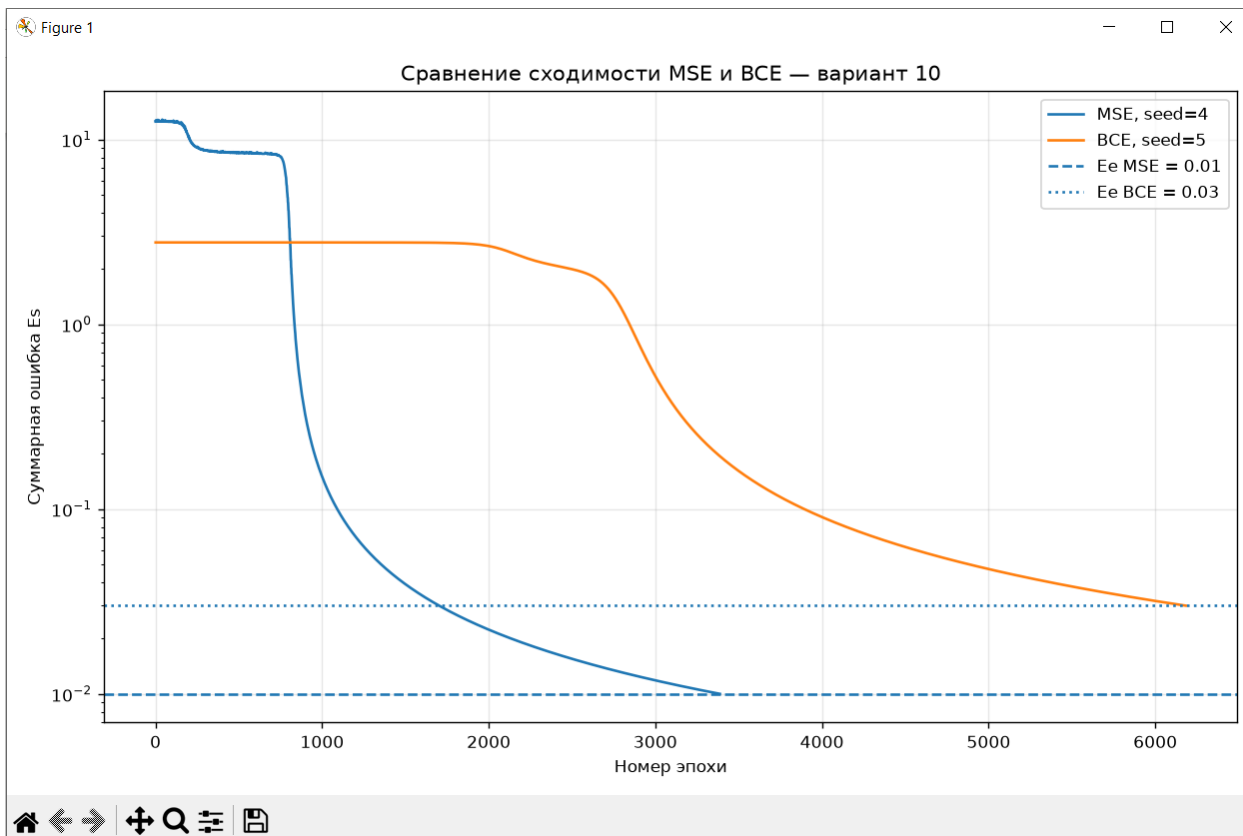
plt.show()

# После закрытия графиков можно
# проверить собственные пары A, B.
interactive_mode(
    bce_rep["model"]
)

if __name__ == "__main__":
    main()

```

Результат программы:



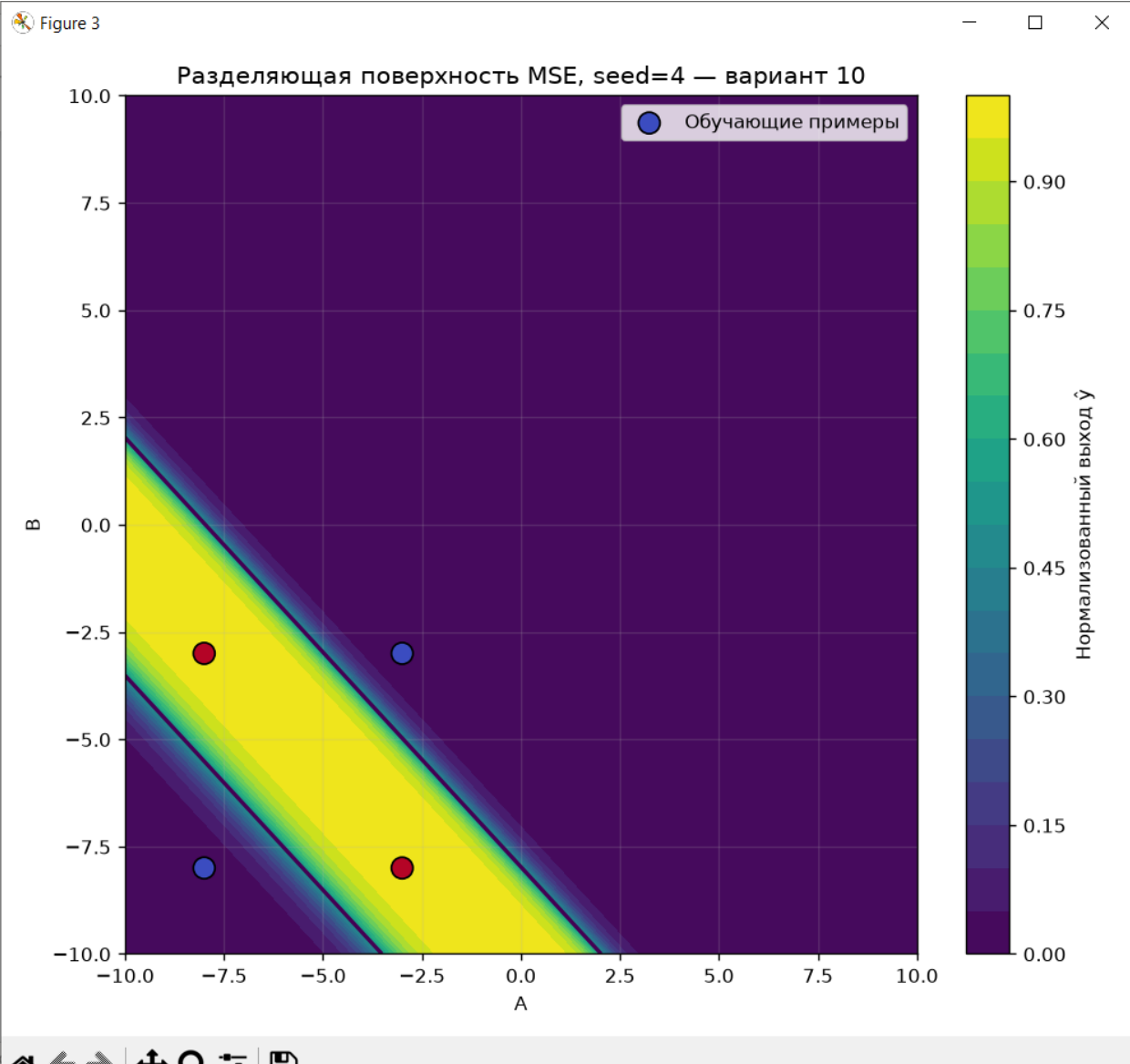
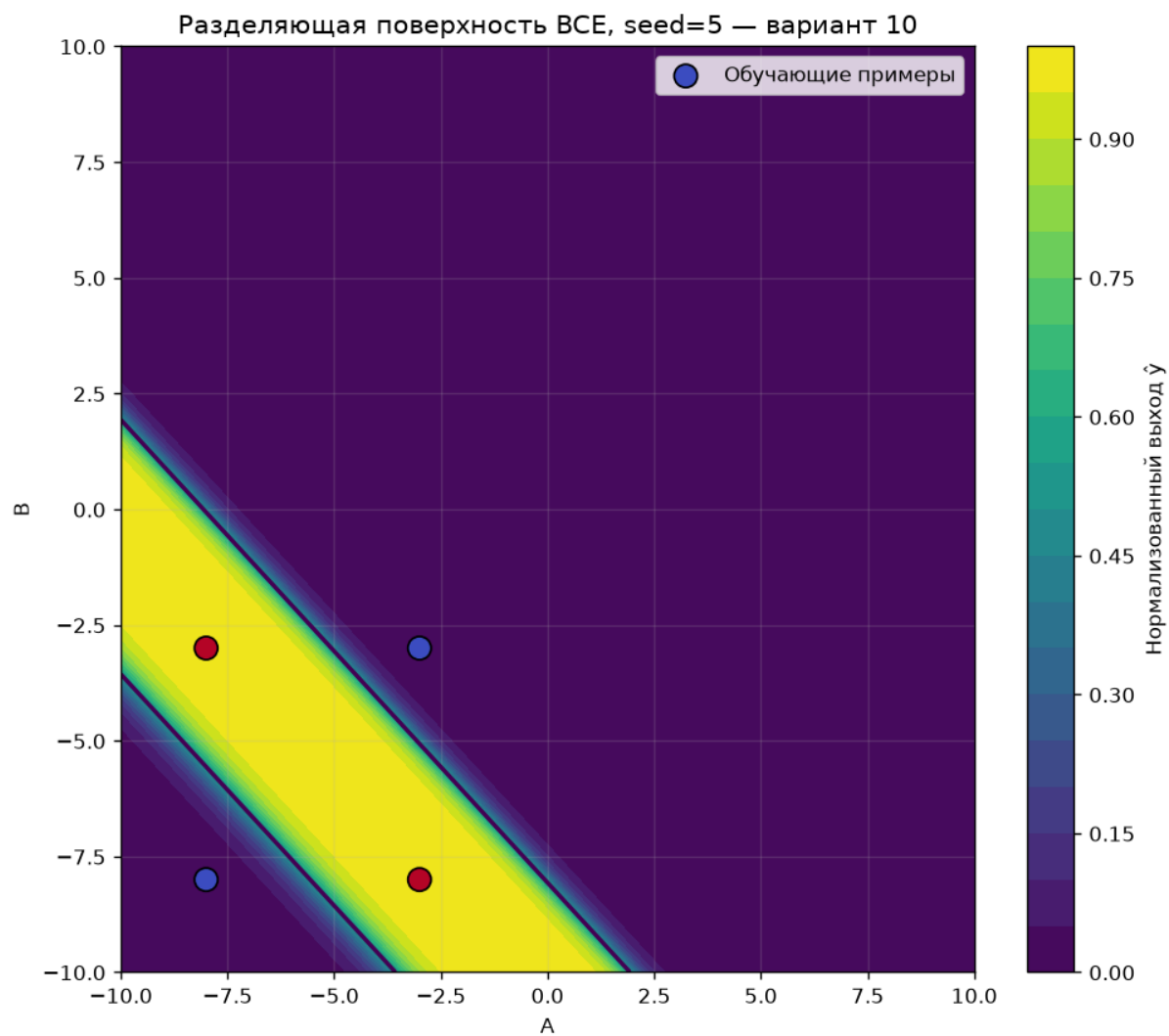
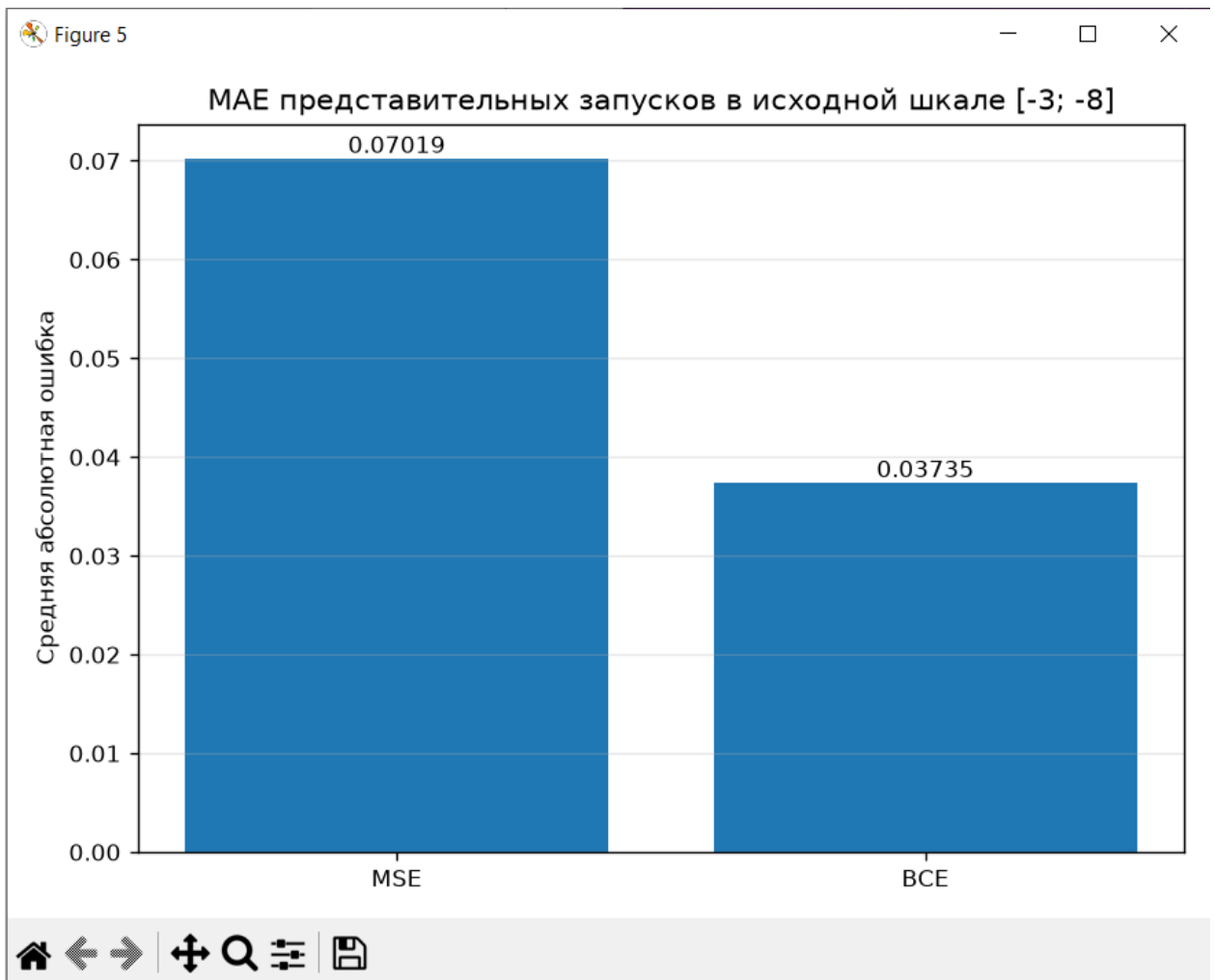


Figure 4





=====

=====

ЛАБОРАТОРНАЯ РАБОТА №2 — ВАРИАНТ 10

$c_0 = -3$, $c_1 = -8$

=====

=====

Конфигурация А: MSE в исходной шкале [-3; -8]

seed= 1 | эпох= 4224 | $E_s=0.009996$ | accuracy=1.00 | MAE=0.070064 | сходимость=ДА

seed= 4 | эпох= 3387 | $E_s=0.009997$ | accuracy=1.00 | MAE=0.070192 | сходимость=ДА

seed= 5 | эпох= 3284 | $E_s=0.009996$ | accuracy=1.00 | MAE=0.070212 | сходимость=ДА

seed= 6 | эпох= 3055 | $E_s=0.009999$ | accuracy=1.00 | MAE=0.070279 | сходимость=ДА

seed= 7 | эпох= 3564 | $E_s=0.009999$ | accuracy=1.00 | MAE=0.070161 | сходимость=ДА

Конфигурация В: ВСЕ по нормализованным целям 0/1

seed= 1 | эпох= 6001 | Es=0.030000 | accuracy=1.00 | MAE=0.037352 | сходимость=ДА

seed= 4 | эпох= 6224 | Es=0.029995 | accuracy=1.00 | MAE=0.037347 | сходимость=ДА

seed= 5 | эпох= 6183 | Es=0.029999 | accuracy=1.00 | MAE=0.037351 | сходимость=ДА

seed= 6 | эпох= 6011 | Es=0.029995 | accuracy=1.00 | MAE=0.037346 | сходимость=ДА

seed= 7 | эпох= 6191 | Es=0.029998 | accuracy=1.00 | MAE=0.037350 | сходимость=ДА

=====

СВОДКА ПО 5 ЗАПУСКАМ

=====

MSE:

Достигли критерия: 5 из 5

Минимум эпох: 3055

Максимум эпох: 4224

Среднее эпох: 3502.80

Разброс: 1169

Средняя ассурасу: 1.0000

Средняя MAE: 0.070181

ВСЕ:

Достигли критерия: 5 из 5

Минимум эпох: 6001

Максимум эпох: 6224

Среднее эпох: 6122.00

Разброс: 223

Средняя ассурасу: 1.0000

Средняя MAE: 0.037349

Представительный запуск MSE

A	B	Цель	$\hat{y}$ norm	$\hat{y}$ real	Класс
-3.0	-3.0	-3.0	0.0124	-3.0622	-3.0
-3.0	-8.0	-8.0	0.9866	-7.9330	-8.0
-8.0	-3.0	-8.0	0.9866	-7.9330	-8.0
-8.0	-8.0	-3.0	0.0169	-3.0845	-3.0

Представительный запуск BCE

A	B	Цель	$\hat{y}$ norm	$\hat{y}$ real	Класс
-3.0	-3.0	-3.0	0.0061	-3.0304	-3.0
-3.0	-8.0	-8.0	0.9933	-7.9664	-8.0
-8.0	-3.0	-8.0	0.9933	-7.9664	-8.0
-8.0	-8.0	-3.0	0.0104	-3.0518	-3.0

=====

РЕЖИМ ФУНКЦИОНИРОВАНИЯ — BCE — ВАРИАНТ 10

=====

A= -3.00, B= -3.00 | $\hat{y}$ =0.0061 | y_{real} =-3.0304 | ближайший класс=-3.0
A= -3.00, B= -8.00 | $\hat{y}$ =0.9933 | y_{real} =-7.9664 | ближайший класс=-8.0
A= -8.00, B= -3.00 | $\hat{y}$ =0.9933 | y_{real} =-7.9664 | ближайший класс=-8.0
A= -8.00, B= -8.00 | $\hat{y}$ =0.0104 | y_{real} =-3.0518 | ближайший класс=-3.0

$A = -5.50, B = -5.50 \mid \hat{y} = 0.9933 \mid y_{\text{real}} = -7.9664 \mid \text{ближайший класс} = -8.0$

$A = -2.00, B = -7.00 \mid \hat{y} = 0.9651 \mid y_{\text{real}} = -7.8257 \mid \text{ближайший класс} = -8.0$

$A = -9.00, B = -4.00 \mid \hat{y} = 0.8375 \mid y_{\text{real}} = -7.1873 \mid \text{ближайший класс} = -8.0$

1. **Сходимость MSE и BCE.**

Обе функции потерь успешно обучили сеть. MSE сошлась быстрее, чем BCE: в среднем ей потребовалось меньше эпох.

2. **Устойчивость к случайной инициализации.**

Обе конфигурации успешно сошлись во всех 5 запусках. BCE показала меньший разброс по числу эпох, поэтому оказалась более устойчивой к выбору начальных весов.

3. **Разделяющая поверхность MSE.**

Сеть с MSE правильно разделила все четыре примера XOR. Полученная граница имеет нелинейную форму, что подтверждает необходимость скрытого слоя.

4. **Разделяющая поверхность BCE.**

Сеть с BCE также правильно разделила классы. Форма разделяющей поверхности похожа на MSE, поэтому обе функции потерь позволяют решить задачу XOR.

5. **Точность восстановления исходной шкалы.**

BCE показала меньшую среднюю абсолютную ошибку, чем MSE. Следовательно, BCE точнее восстановила исходные значения классов, хотя для обучения ей потребовалось больше эпох.

Вывод: В ходе лабораторной работы был исследован многослойный персептрон архитектуры 2–2–1 для решения задачи XOR варианта 10 с числовыми классами $c_0 = -3$ и $c_1 = -8$. Были реализованы и сравнены две функции потерь: MSE и Binary Cross-Entropy.

Для использования BCE исходные классы были нормализованы: значению -3 соответствовала метка 0, а значению -8 — метка 1. После обучения выход сети преобразовывался обратно в исходную шкалу по формуле $y_{\text{real}} = -3 - 5\hat{y}$.

Обе конфигурации продемонстрировали устойчивую сходимость: во всех пяти запусках для MSE и BCE был достигнут заданный критерий остановки, а точность классификации составила 100%.

MSE показала более высокую скорость обучения. Среднее число эпох до достижения критерия составило 3502,8 для MSE и 6122 для BCE. При этом BCE оказалась более устойчивой к изменению случайной инициализации: разброс числа эпох составил 223, тогда как для MSE — 1169.

По точности восстановления исходной числовой шкалы лучший результат показала BCE. Средняя абсолютная ошибка составила 0,037349 для BCE и 0,070181 для MSE. Таким образом, несмотря на более медленное обучение, BCE обеспечила более точное восстановление исходных значений классов.

Разделяющие поверхности обеих конфигураций имеют сходную форму и корректно отражают структуру XOR. Две комбинации класса $c_1 = -8$ располагаются внутри сформированной нелинейной области, а комбинации класса $c_0 = -3$ — за её пределами.

Полученные результаты подтверждают, что XOR остаётся линейно неразделимой задачей независимо от конкретных числовых значений, используемых для кодирования классов. Выбор функции потерь влияет на скорость, устойчивость и точность обучения, но не изменяет необходимость использования скрытого слоя многослойного персептрона.

Таким образом, для варианта 10 MSE оказалась быстрее, а BCE — стабильнее по числу эпох и точнее по восстановлению исходной шкалы.